

Forecasting and Anomaly Detection with Deep Learning for Effluent Phosphorus on The BSM2 Benchmark

Jehan Kanaganathan

(Dated: June 5, 2026)

This study investigate two complementary deep learning approaches for monitoring effluent phosphate in a simulated wastewater treatment plant. A short-horizon forecasting of effluent phosphorus model, and a unsupervised model of normal plant operation for anomaly detection. Three sequence architectures (LSTM, GRU, Transformer) are compared for forecasting on the Benchmark Simulation Model No.2. All three beat a persistence baseline by roughly 80%, with the LSTM performing best (RMSE = 0.253 mg L⁻¹). A sequence variational autoencoder, trained on the same input windows, learns to reconstruct normal plant behavior with low error and produces a per-window anomaly score from the maximum reconstruction error. Integrating the two models on a test set, the anomaly score is positively correlated with the forecaster’s prediction error (Pearson $r = +0.647$, Spearman $r = +0.492$), and abstaining on the 20% of windows with the highest anomaly score reduces the forecaster’s RMSE on the remaining windows by 14.8%. The two models, trained independently and for different objectives, agree on which windows are difficult, demonstrating that an unsupervised generative model can serve as a useful confidence signal for a separately trained forecaster.

I. INTRODUCTION

Wastewater treatment plants are essential infrastructure for protecting natural water bodies from nutrient pollution. Phosphorus is particularly damaging, as elevated concentration can cause eutrophication, algal blooms and oxygen depletion (Smil, 2000). Effluent phosphorus is therefore subject to strict regulatory limits across Europe (European Council, 2024). Meeting these limits is challenging because effluent phosphorus in a treatment plant is often the end result of a complex removal process, which includes biological processes across multiple reactors, driven by variable loading, temperature and microbial dynamics (Henze *et al.*, 1999). Most modern wastewater treatment plants are heavily instrumented, with sensors continuously measuring flow rates, dissolved oxygen, nutrient concentrations, and temperature across various parts of the treatment plant. This data, combined with modern deep learning methods, opens up new possibilities for predictive monitoring and control of wastewater operations. This allows the operators to anticipate plant behavior rather than respond to real-time measurements.

This study investigates two complementary uses of deep learning on plant state time series. The first is short-horizon forecasting of effluent phosphorus, which can be used for predictive control before regulatory limits are reached. The second is unsupervised anomaly detection, modelling the distribution of normal plant behavior and flagging windows that fall outside it. Both models uses the same input of a window of recent plant state. This shared input motivates the central question explored in this study: Can the two deep learning models, with independent training, be combined so that anomaly signals informs when to use the forecast?

The contributions are:

- A comparison of three sequence architectures (LSTM, GRU, Transformer) for forecasting effluent phosphate on the Benchmark Simulation Model No. 2 (BSM2) (Gernaey *et al.*, 2014), a widely used municipal wastewater plant simulation.
- A sequence variational autoencoder (Kingma and Welling, 2013) for unsupervised modelling of normal plant operation.
- An integration analysis showing that the autoencoder’s anomaly score correlates with the forecaster’s prediction error and can be used to flag windows where the forecast is unreliable.

The report is structured as follows. Section II presents the theoretical foundations of the forecasting and generative models, the integration framework, the BSM2 simulation and the variables used, and the implementation details. Section III reports and discusses the results of the three forecasting architectures, the variational autoencoder’s training behaviour and reconstruction quality, and the integration analysis, and discusses the limitations of the present work and directions for future research. Section IV summarises the report.

II. THEORY AND METHODS

A. Problem formulation

This study considers a multivariate time series sampled from a simulated wastewater treatment plant. At each time step t , the plant state is represented by a vector $\mathbf{x}_t \in \mathbb{R}^V$ containing $V = 19$ observations. The effluent phosphate concentration S_{PO_4} , together with 18

upstream variables describes influent values and the internal states of treatment plant tanks and reactors, and are sampled at a resolution of $\Delta t = 15$ minutes.

Two problems are formulated using this time series. The first is multi-step-ahead forecasting of the effluent phosphate concentration. Given a past window of $L = 48$ observations (twelve hours of recent plant state values), the goal is to predict the target trajectory $H = 24$ steps ahead (six hours of future effluent values). This problem is approached using discriminative methods, where three sequence architectures (Section II.B) each learn a direct mapping from the past window to the future target values.

The second problem is the unsupervised modelling of the plant state. Rather than predicting forward in time, the goal is to learn the distribution of plant-state windows that occur under normal operating conditions. Once such a distribution is learned, windows assigned low probability from the model can be flagged as an anomaly, providing an unsupervised signal for sensor faults or process disturbances. This problem is approached using a sequence variational autoencoder (Section II.C).

B. Discriminative models

Three sequence architectures are compared based on the forecasting problem defined earlier: a Long Short-Term Memory network (LSTM), a Gated Recurrent Unit network (GRU), and a Transformer encoder. All three share the same input/output with a past window of $L = 48$ steps in, a future trajectory of $H = 24$ steps out, and differ only in how they encode the past window into a context vector. This shared structure allows direct comparison of the three architectures without confounding factors from differing input formats or output heads.

1. Long Short-Term Memory

Recurrent neural networks (RNNs) process sequences one element at a time, maintaining a hidden state that summarizes everything seen so far. In their basic form, recurrent networks struggle to retain information across long sequences, as gradients propagated back through many time steps tend to either vanish or explode, making it difficult to learn dependencies that span over a long range (Raschka *et al.*, 2022). The Long Short-Term Memory network (Hochreiter and Schmidhuber, 1997) was introduced to overcome these limitations by extending the recurrent unit with a separate cell state and three gates that regulate the flow of information through it.

At each time step t , an LSTM cell receives the current input $\mathbf{x}_t$ and the previous hidden state $\mathbf{h}_{t-1}$, and computes:

$$\mathbf{i}_t = \sigma(W_i \mathbf{x}_t + U_i \mathbf{h}_{t-1} + \mathbf{b}_i) \quad (\text{input gate}) \quad (1)$$

$$\mathbf{f}_t = \sigma(W_f \mathbf{x}_t + U_f \mathbf{h}_{t-1} + \mathbf{b}_f) \quad (\text{forget gate}) \quad (2)$$

$$\mathbf{o}_t = \sigma(W_o \mathbf{x}_t + U_o \mathbf{h}_{t-1} + \mathbf{b}_o) \quad (\text{output gate}) \quad (3)$$

$$\tilde{\mathbf{C}}_t = \tanh(W_c \mathbf{x}_t + U_c \mathbf{h}_{t-1} + \mathbf{b}_c) \quad (\text{candidate cell}) \quad (4)$$

$$\mathbf{C}_t = (\mathbf{f}_t \odot \mathbf{C}_{t-1}) \oplus (\mathbf{i}_t \odot \tilde{\mathbf{C}}_t) \quad (\text{cell state}) \quad (5)$$

$$\mathbf{h}_t = \mathbf{o}_t \odot \tanh(\mathbf{C}_t) \quad (\text{hidden state}). \quad (6)$$

where σ is the sigmoid function, $\odot$ denotes element-wise multiplication, $\oplus$ denotes element-wise summation, and the matrices W, U and bias vectors $\mathbf{b}$ are the learnable parameters of the cell.

The cell state $\mathbf{c}_t$ acts as a long-term memory that is updated additively rather than multiplicatively, which prevents the rapid decay of gradients seen in basic recurrent networks (Raschka *et al.*, 2022). The forget gate $\mathbf{f}_t$ decides what fraction of the previous cell state to retain, the input gate $\mathbf{i}_t$ controls how much of the new candidate $\tilde{\mathbf{C}}_t$ to add, and the output gate $\mathbf{o}_t$ determines what part of the cell state is exposed as the hidden state $\mathbf{h}_t$ passed to subsequent layers (Gers *et al.*, 2000).

The forecaster used in this project is bidirectional, where two LSTMs process the input window in opposite directions, one from $t = 1$ to $t = L$ and the other from $t = L$ to $t = 1$, and their hidden states are concatenated at each step (Schuster and Paliwal, 1997). The advantage in this setting is that every step has access to both past and future context (in the input window), allowing the model to identify temporal patterns regardless of where they occur. After the bidirectional pass, the concatenated hidden state at the final time step serves as the context vector summarising the past window. This context vector is passed to the forecast head.

2. Gated Recurrent Unit

The Gated Recurrent Unit (Cho *et al.*, 2014) is an alternative to the LSTM with fewer parameters and simpler structure. It replaces the input and forget gates with a reset and update gate, and removes the separate cell state. At each step, the GRU computes:

$$\mathbf{r}_t = \sigma(W_r \mathbf{x}_t + U_r \mathbf{h}_{t-1} + \mathbf{b}_r) \quad (\text{reset gate}) \quad (7)$$

$$\mathbf{z}_t = \sigma(W_z \mathbf{x}_t + U_z \mathbf{h}_{t-1} + \mathbf{b}_z) \quad (\text{update gate}) \quad (8)$$

$$\tilde{\mathbf{h}}_t = \tanh(W \mathbf{x}_t + U(\mathbf{r}_t \odot \mathbf{h}_{t-1}) + \mathbf{b}) \quad (\text{candidate state}) \quad (9)$$

$$\mathbf{h}_t = (1 - \mathbf{z}_t) \mathbf{h}_{t-1} + \mathbf{z}_t \tilde{\mathbf{h}}_t \quad (\text{hidden state}). \quad (10)$$

The reset gate $\mathbf{r}_t$ controls how much of the previous hidden state contributes to the candidate state. The update gate $\mathbf{z}_t$ interpolates between the previous hidden

state and the new candidate state. Even though the GRU has fewer parameters than the LSTM, it has been showed to perform similarly on a range of different tasks (Chung *et al.*, 2014). It is included here to test its performance against the LSTM and Transformer on wastewater forecasting tasks. Similar to the LSTM in this project, the GRU forecaster is bidirectional, sharing the same pass structure.

3. Transformer

The Transformer (Vaswani *et al.*, 2017) differs from the LSTM and GRU in structure. Instead of processing the sequence one step at a time, every position interacts with every other position simultaneously through a mechanism called self-attention. The advantage is that any two time steps are connected by a single layer of computation, regardless of their distance in the sequence.

The core operation is scaled dot-product attention. Each input position is projected into three vectors: a query Q , a key K , and a value V . The attention output is computed as:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right)V, \quad (11)$$

where d_k is the dimension of the vectors. The product $QK^\top$ measures the similarity between each query and each key, the softmax converts these similarities into weights, and the result is a weighted combination of the values. Multi-head attention applies this operation in parallel with independent projections, allowing different heads to focus on different temporal patterns.

In the forecaster used in this project, the input variables at each time step are first linearly projected to a d -dimensional representation. Because self-attention has no inherent sense of order, a learned positional embedding is added to each projected step (Vaswani *et al.*, 2017). The result is passed through a stack of Transformer encoder layers, each combining multi-head self-attention with a small feedforward network. The output of the final layer is averaged across the time dimension to produce a single context vector, which is then passed to the forecast head.

4. Forecast head and training objective

All three encoders, the LSTM, GRU, and Transformer, produce a single fixed-size context vector summarising the past window. This vector is passed through one linear layer that outputs all H future values in a single forward pass. This is known as direct multi-step forecasting, where all H future values are predicted at once, rather

than one step at the time which feeds into the next prediction. The alternative, where the steps are generated one by one, causes errors to compound over the horizon (Taieb and Atiya, 2016).

All three models are trained with mean squared error (MSE) between predicted and observed targets on min-max-scaled data. Optimisation uses the Adam optimiser (Kingma and Ba, 2014) with weight decay and a learning rate scheduler that halves the rate when the validation loss plateaus. Training stops early when the validation loss does not improve for a fixed number of epochs. The full set of hyperparameters used, with hidden size, number of layers, dropout, learning rate, and batch size, is given in Section II.F, along with the hyperparameter search procedure.

C. Generative model: Variational autoencoder

The second model in this project approaches the same plant data from a different angle. Rather than predicting forward in time, it learns the distribution of normal plant-state windows and flags windows that fall outside this distribution as potential anomalies. This is an unsupervised problem where the model is never told what the anomalies are, rather it learns the structure of normal operation by reconstructing the data, and the anomalies emerge as poorly reconstructed windows. The framework used here is the variational autoencoder (VAE), first introduced by Kingma and Welling (2013). This is a neural network architecture that learns to compress inputs into structured low-dimensional representation and reconstruct from that representation. Because this representation follows a probability distribution, the VEA is a generative model. It learns the distribution from the training data and new samples can be generated from that distribution. In this project, the fact that the inputs that the model reconstruct poorly is unlikely based on the learned distribution, it is used to detect the anomalies.

1. Setup

Each past window $\mathbf{x} \in \mathbb{R}^{L \times V}$ is assumed to be generated from a low-dimensional unobserved latent variable $\mathbf{h} \in \mathbb{R}^{D_h}$, which captures the essential underlying state of the plant in compressed form. The latent is drawn from a standard Gaussian prior $p(\mathbf{h}) = \mathcal{N}(\mathbf{0}, \mathbf{I})$, and the observation is generated by a neural network decoder with parameters θ , giving the conditional distribution $p_\theta(\mathbf{x} | \mathbf{h})$.

Training would ideally maximise the marginal likelihood $p(\mathbf{x}; \theta)$, but this requires integrating over all possible latent vectors and is intractable. The VAE works around this by introducing a second neural network, which is the encoder that learns to approximate the true

posterior with a tractable Gaussian, given as:

$$q_\phi(\mathbf{h} \mid \mathbf{x}) = \mathcal{N}(\mathbf{h}; \boldsymbol{\mu}_\phi(\mathbf{x}), \boldsymbol{\sigma}_\phi^2(\mathbf{x})\mathbf{I}), \quad (12)$$

where $\boldsymbol{\mu}_\phi$ and $\log \boldsymbol{\sigma}_\phi^2$ are the outputs of a neural network with parameters ϕ and the covariance is restricted to be diagonal for cheaper computation.

2. The training objective

The encoder and decoder are trained together by maximising a lower bound on the log marginal likelihood, known as the Evidence Lower Bound (ELBO) (Kingma and Welling, 2019), given as:

$$\mathcal{L}_{\text{ELBO}}(\phi, \theta; \mathbf{x}) = \mathbb{E}_{q_\phi(\mathbf{h} \mid \mathbf{x})}[\log p_\theta(\mathbf{x} \mid \mathbf{h})] - D_{\text{KL}}(q_\phi(\mathbf{h} \mid \mathbf{x}) \parallel p(\mathbf{h})). \quad (13)$$

The first term, the reconstruction term, is large when the decoder accurately reproduces the input $\mathbf{x}$ from a latent sample drawn by the encoder. The second term, the prior-matching term, pulls the encoder distribution towards the prior $p(\mathbf{h})$ and prevents it from collapsing to a narrow distribution fitted to individual training examples. Maximising the ELBO therefore pushes the reconstruction term up and the prior-matching term down. In practice we use the negative ELBO during training, so the reconstruction term becomes a reconstruction loss to minimise (MSE for a Gaussian decoder).

3. Closed-form KL and the reparameterisation trick

The two terms of the ELBO are computed differently during training. The prior-matching term has a closed-form expression because both the encoder distribution and the prior are diagonal Gaussians (Kingma and Welling, 2019), given as:

$$\begin{aligned} D_{\text{KL}}(q_\phi(\mathbf{h} \mid \mathbf{x}) \parallel p(\mathbf{h})) \\ = -\frac{1}{2} \sum_{d=1}^{D_h} (1 + \log \sigma_{\phi,d}^2 - \mu_{\phi,d}^2 - \sigma_{\phi,d}^2). \end{aligned} \quad (14)$$

This depends only on the encoder's mean and log-variance outputs and can be computed directly without sampling.

The reconstruction term is an expectation value over the decoder's distribution that has no closed form. It is approximated using one Monte Carlo sample, where $\mathbf{h}$ is drawn from $q_\phi(\mathbf{h} \mid \mathbf{x})$ and passed through the decoder. The resulting reconstruction error is used as an estimate for the expectation value.

Drawing $\mathbf{h}$ from a distribution whose parameters depend on ϕ is a stochastic operation, and gradients cannot

flow back through it to update the encoder. The reparameterisation trick solves this by rewriting the sample as a deterministic function of the encoder output and a noise variable (Kingma and Welling, 2019), given as:

$$\mathbf{h} = \boldsymbol{\mu}_\phi(\mathbf{x}) + \boldsymbol{\sigma}_\phi(\mathbf{x}) \odot \boldsymbol{\epsilon}, \quad \boldsymbol{\epsilon} \sim \mathcal{N}(\mathbf{0}, \mathbf{I}), \quad (15)$$

where $\odot$ denotes element-wise multiplication. Now the randomness is in $\boldsymbol{\epsilon}$, which has no learnable parameters, and the gradients can flow through.

4. The β -VAE objective

For anomaly detection, the goal is to reconstruct normal plant-state windows as good as possible, so that unusual windows stand out due to their large reconstruction error. The standard ELBO weights the reconstruction term and the prior-matching term equally, which is normal for generative sampling, but not necessarily for reconstruction performance. In order to change the balance, we use the β -VAE objective (Higgins *et al.*, 2017), which scales the prior-matching term. This is given as:

$$\mathcal{L}_\beta(\phi, \theta; \mathbf{x}) = -\mathbb{E}_{q_\phi}[\log p_\theta(\mathbf{x} \mid \mathbf{h})] + \beta \cdot D_{\text{KL}}(q_\phi \parallel p(\mathbf{h})). \quad (16)$$

With $\beta = 1$ this becomes the negative ELBO of Equation (13). Choosing $\beta < 1$ reduces the weight of the prior-matching term.

An issue when training VAEs is posterior collapse, where the decoder learns to ignore the latent variable and KL term drops to zero. To avoid this we increase β from zero to its target value over the first few training epochs, allowing the model to learn as much as it can before increasing the posterior term pressure (Bowman *et al.*, 2016).

5. Architecture

The model architecture used in this project contains an encoder which is a bidirectional LSTM (BiLSTM). This processes past windows in both directions and produces hidden states at each of L time steps. These are pooled into a single fixed-size representation by concatenating their mean with the final state. Two parallel linear layers then produce the mean $\boldsymbol{\mu}_\phi$ and log-variance $\log \boldsymbol{\sigma}_\phi^2$ of the approximate posterior in Equation (12).

The decoder takes a sampled latent $\mathbf{h}$ and projects it back to the hidden dimension before repeating this projection across all L time steps. It adds a learned positional embedding so that each step has a distinct position in time (Vaswani *et al.*, 2017). The resulting sequence is processed by an LSTM and projected to the V output variables to produce the reconstruction $\hat{\mathbf{x}}$.

Compared to a traditional VAE, the encoder and decoder used in the project are sequence models (BiLSTM and LSTM) in order to capture the temporal patterns in the input data.

6. Anomaly scoring

Once trained, the VAE assigns each input window a reconstructed value $\hat{\mathbf{x}}$, and the squared error between $\mathbf{x}$ and $\hat{\mathbf{x}}$ measures how well the model recognises the window as a sample from the normal-operation distribution. We summarise this per-window into a single anomaly score, given as (An and Cho, 2015):

$$s(\mathbf{x}) = \max_{t,v} (x_{t,v} - \hat{x}_{t,v})^2, \quad (17)$$

where the maximum is taken over all time steps t and all variables v in the window. Higher scores indicate larger discrepancies between the input and the reconstruction, and therefore a higher likelihood that the window contains anomalies.

We use the maximum rather than the sum or the mean, as one high score in one variable can get averaged out by a sum over $L \times V$ elements, losing the strongest signal for an anomaly. To reduce the variance from the stochastic sampling, the squared error is averaged over four draws from the encoder before the maximum is taken.

D. Integration

The forecaster and the VAE are trained independently and solve different problems. The forecaster predicts future effluent phosphorus steps and the VAE models the distribution of the recent past. They do however share the same input, with both models receiving the L -step past window of the plant state. This makes it possible to do further analysis using the two models and a test set, by asking the questions: do the models agree on which windows are difficult? If the VAE flags a window as unusual, is the forecaster more likely to produce a poor forecast on the same window?

The test set uses a previously not used portion of the data in non-overlapping windows. Each window contributes with the past scaled window for the forecaster (min-max scaling), the same past window scaled for the VAE (standard scaler), and the future target trajectory used to evaluate the forecaster. Both models run on each test window and metrics are calculated using root mean-squared error (RMSE) for the forecaster and the anomaly score for the VAE. The metrics are then used in the analysis described below.

1. Score correlation

The first analysis tests whether the two models agree on which windows are difficult. Each window contributes to one point on a scatter plot with the VAE anomaly score on one axis and the forecaster RMSE on the other. Both Pearson and Spearman correlations coefficients are reported, where the Pearson is more sensitive to outliers. A positive correlation indicates that windows the VAE flags as unusual tends to be the windows the forecaster gets wrong, providing evidence that two models pick up the same properties of the data despite being trained for different objectives.

2. Conditional forecast accuracy

The second analysis translates the score correlation into an operational measure. Test windows are sorted by VAE anomaly score and divided into five equally sized bins, ranging from cleanest (lowest anomaly score) to dirtiest (highest anomaly score). Then the forecaster RMSE is calculated for each bin. If the RMSE clearly rises from cleanest to dirtiest, it can be confirmed that the VAE provides useful ranking of forecasting difficulty. In an operational setting where a forecaster already exists, a working VAE can tell when to trust the forecaster.

We also report a selective prediction curve, which computes the forecast RMSE after discarding the top f percent of the dirtiest windows. This tells us how much accuracy can be gained by abstaining on the windows the VAE considers anomalous. A clear drop in RMSE as f increases indicates that the signal is useful, while a flat curve indicates that the anomaly score is not informative.

E. Data

This project uses simulated wastewater treatment plant data from the Benchmark Simulation Model No. 2 (BSM2) (Jeppsson *et al.*, 2007). BSM2 provides a clean, fully observable, and reproducible plant state, making it well-suited for evaluating forecasting and anomaly detection. The project was originally built around the use of real plant data from a wastewater treatment plant located in Hamar, Norway, but this was ultimately not used due to substantial preprocessing needs.

1. The BSM2 benchmark

BSM2 is the international benchmark for simulating municipal wastewater treatment plants and is used widely for testing control and monitoring strategies. It simulates a complete plant comprising of a primary clarifier, an anaerobic-anoxic-aerobic activated sludge reactor train, a secondary clarifier, a sludge thickener, an

anaerobic digester, a dewatering unit, and a storage tank (Solon *et al.*, 2017). The biological dynamics are governed by the Activated Sludge Model No. 2d (ASM2d) (Henze *et al.*, 1999), which models the carbon, nitrogen, and phosphorus processes including enhanced biological phosphorus removal. The simulation is produced using realistic diurnal, weekly, seasonal influent patterns, and rain events, and calculates the full plant state as time series, which is every component concentration in every reactor (Gernaey *et al.*, 2014; Jeppsson *et al.*, 2007).

The data gained from the simulation has a resolution of $\Delta t = 15$ over a one-year period (364 days). The variable to be predicted and monitored is the effluent phosphate concentration SP_{O_4} . The variable was first chosen due to being the observed effluent variable in the real plant previously headlining this project.

2. Variable selection

Of the variables produced by the simulation, $V = 19$ are used as the input features in both models. The selection is guided by underlying processes driving biological phosphorus removal, such as influent loading, anaerobic and aerobic process variables (Henze *et al.*, 1999). Although the full set of ASM2d variables was available for every reactor, a limited set was chosen initially for comparison with available observed real plant variables. Furthermore, using a limited set of simulated variables, provides a somewhat realistic plant monitoring structure as well as reducing model complexity and computational cost. The selected variables are listed below:

- **Target:** effluent phosphate.
- **Influent:** flow rate, temperature, phosphate, ammonium, fermentable substrate, fermentation products, slowly biodegradable substrate, total suspended solids.
- **Anaerobic reactor 1:** phosphate, polyphosphate-accumulating organism biomass, stored polyphosphate, fermentation products.
- **Aerobic reactors 5 and 7:** dissolved oxygen, phosphate, ammonium, total suspended solids.

3. Splits and windowing

The time series is split chronologically into training (75%), validation (10%), and test (15%) sets. The split is ordered in time, without shuffling, so to not allow the models to see information from future time points during training, as it would be in real-world use.

Two different scalars are applied to the data. The forecaster uses min-max scaling, mapping each variable to the interval $[0, 1]$, because early runs showed better results than with standard scaler. The VAE uses standard scalar (zero mean, unit variance per variable). Both scalar

parameters are computed using just the training data and applied to scale training, validation and testing.

Both models use fixed-length windows of $L = 48$ time steps (twelve hours). For the forecaster each training pair consists of a past window paired with the next $H = 24$ steps (six hours) of effluent phosphate target. For the VAE, each window is paired with itself. Training and validation windows are extracted with a stride of $L/4$, which gives a good coverage of the training distribution. Test windows are extracted with a stride of L to not overlap. The length of the window is chosen to capture diurnal wastewater patterns while still having a relatively low number of steps.

F. Implementation and reproducibility

All models are implemented using PyTorch (Paszke *et al.*, 2019). Data handling uses pandas, NumPy, and scikit-learn. Hyperparameter search is conducted using Optuna (Akiba *et al.*, 2019).

The project code is organised into three python notebooks that share a common configuration block specifying the window length, horizon, variable list, target feature, and split ratios. The first notebook trains the forecaster, the second trains the VAE, and the third loads both trained models using saved checkpoints and runs the integration analysis from II.D. The shared configuration ensures that the two models have aligned test windows.

The three forecasting architectures (LSTM, GRU, and Transformer) are tuned independently with Optuna using the validation MSE as the objective. The search parameters include, hidden size, number of layers, dropout, learning rate, and (for the Transformer) the number of attention heads. The VAE is tuned separately with validation reconstruction loss as the objective. This includes a guard term that penalises trials where the KL divergence collapses to zero. All Optuna runs are saved in a SQLite database so that the search can be resumed or inspected after completion. The selected hyperparameters for each model are reported in III.

All training runs use the Adam optimiser (Kingma and Ba, 2014) with weight decay, a learning rate scheduler that halves the rate when the validation loss plateaus, and early stopping when the validation loss does not improve for a fixed number of epochs. Random seeds are fixed and the CUDA backend is set to deterministic mode, so the same code produces the same result across runs. Each model is trained and saved together with its full architecture configuration and the scalar statistics that was fitted on the training data set.

As a benchmark for the forecaster, a persistence baseline is used, meaning that forecast at every horizon step is equal to the last observed target value. This is intentionally simple, meaning that a useful forecaster should easily beat it.

TABLE I Tuned hyperparameters for the three forecasting architectures. Common settings across all architectures: batch size = 64, weight decay = 10^{-5} , patience = 25 epochs, maximum epochs = 300.

	LSTM	GRU	Transformer
Hidden size	128	128	32
Layers	1	2	3
Dropout	0.080	0.196	0.053
Learning rate	4.20×10^{-3}	3.38×10^{-3}	4.46×10^{-3}
Attention heads	—	—	4
Optuna trials	100	100	60
Parameters	158,744	417,048	105,080

The complete code, including training scripts, optuna summary, saved checkpoints, and run logs, is available at the project GitHub repository¹.

III. RESULTS AND DISCUSSION

A. Forecasting

1. Tuned architectures and training behaviour

The three forecasting architectures, LSTM, GRU, and Transformer, were independently tuned with Optuna (Akiba *et al.*, 2019) using validation MSE as the objective. Table I reports the hyperparameters selected for each model (for tuning details, see Github repository). The LSTM converged on a single-layer architecture with a hidden size $h = 128$ and a low dropout, suggesting it benefits having a wide but shallow structure while using most of its neurons. The GRU converged on the same width, but with two layers and a higher dropout, letting the layers do more of the work. The transformer converged on a narrow structure while having 3 layers and 4 attention heads. Due to the wide structure and multiple layers, the GRU had a higher parameter count. The LSTM and GRU were tuned with a 100 Optuna trials each, while the Transformer was tuned with 60 trials.

Figure 1 shows the training and validation loss for each architecture over training epochs. All three models converge smoothly to a stable validation loss, and the gap between training and validation curves is relatively, indicating that early stopping and weight decay are effective in controlling overfitting. The LSTM required more epochs to reach its best validation loss than the others, but given all converged smoothly, the number of epochs and the patience could have been reduced.

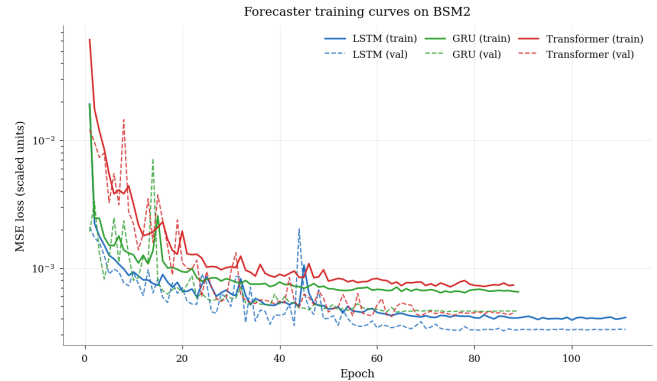


FIG. 1 Forecaster training curves on BSM2. Training loss (continuous) and validation loss (dashed) on a log scale for the three architectures: LSTM (blue), GRU (green), and Transformer (red).

TABLE II Forecast performance on the BSM2 test set. RMSE and MAE are reported in mg L^{-1} for effluent phosphate $SP_{O_4}^e$. Performance compared to the persistence baseline is given in %.

Model	RMSE	MAE	R^2	vs. persistence
LSTM	0.253	0.158	0.987	+80.8%
GRU	0.270	0.172	0.985	+79.5%
Transformer	0.291	0.191	0.983	+77.9%

2. Forecast performance

Table II reports the results from the testing data set for the three architectures alongside the persistence baseline. All three deep learning models beat persistence by a large margin, which shows that the past plant-state window contains substantial information beyond the most recent target value. The difference between the models are small but consistent across all metrics. The LSTM performed the best, and was therefore chosen for the subsequent analysis and integration with the VAE.

Figure 2 shows six example forecasts of the target $SP_{O_4}^e$ for the LSTM, taken for a distribution of per-window RMSE. One the windows with lowest and typical RMSE, the prediction closely tracks the true future trajectory throughout the full 6 hours. On the poor predictions, the prediction still performs reasonably, but the worst performance show that the model struggle with sharp transitions. The baseline trajectory visually confirms how much information the model recovers beyond just using the previous target. Examples forecasts for all model architectures is given in A.1.

The plot of the RMSE by horizon step in Figure 3 provides a view of how forecast accuracy degrades with prediction distance. RMSE grows slow, almost linearly with the horizon for all three architectures, while the persistence baseline grows fast before dropping closer to the end of the horizon. The drop in RMSE for the baseline is

¹ <https://github.com/jekan-lab/FYS9429-Report-Repo.git>

Forecasts on BSM2 — LSTM

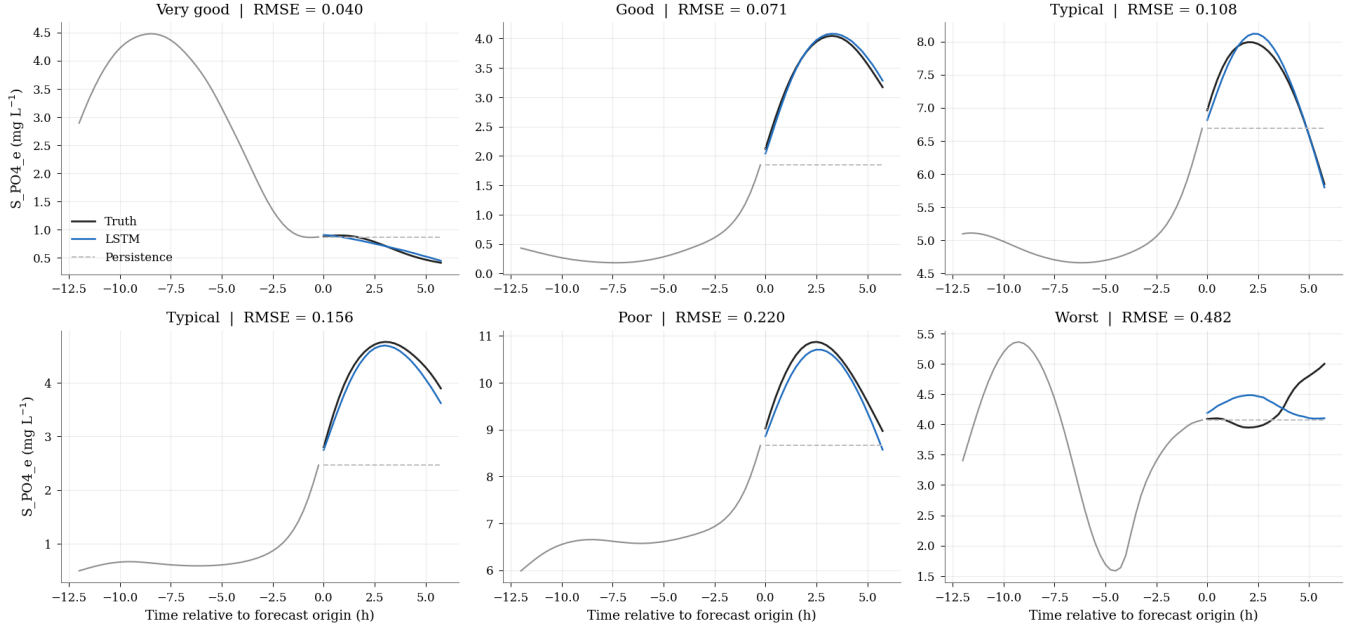


FIG. 2 LSTM forecast trajectories on six representative BSM2 test windows, based on a distribution of per-window RMSE (order is best to worst, left to right and top to bottom). Each panel shows the past 12-hour window (grey), the true future 6-hour trajectory (dark), the LSTM prediction (blue), and the persistence baseline (dashed grey).

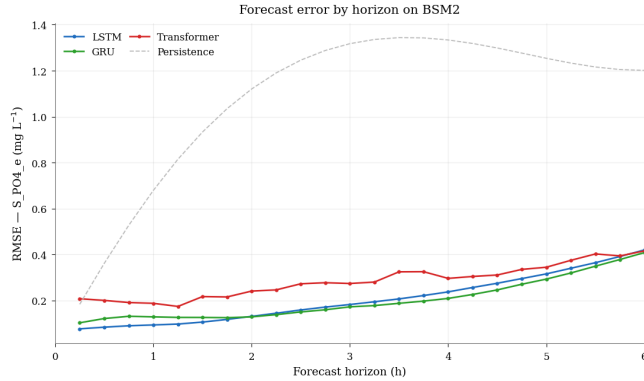


FIG. 3 Forecast error by horizon on BSM2. RMSE for the three architectures (solid, with markers) and persistence (dashed).

probably due the cycles present in the data, where values further in the future might in some cases be more similar to the values at the start of the horizon. The LSTM and GRU degrades smoothly with negligible differences, while the Transformer has a degrading trend with some unusual behavior. This might be due to the model being able to capture some internal dynamics well and others not.

3. Architecture comparison

The three architectures perform within roughly 1% of each other, with no single model being dominant. This suggests that on this dataset, and at this horizon, the choice between LSTM, GRU, and Transformer matters less than the choice to use a deep learning model. All three models are well-suited to capture the diurnal and weekly cycles that affect the phosphorus effluent, and they reach similar performance once tuned.

A possible explanation for the Transformer's slightly weaker performance is sequence length. The Transformer architecture, with direct and distance-independent connections inputs, might be more valuable on longer sequences where RNNs might struggle to retain information across many steps. The Transformer's smaller parameter count reflects the simpler representation it needs to model the windows. The comparison is still useful as a check if attention can capture some structural features the RNNs cannot.

B. Generative modelling - VAE

1. Training behaviour

The VAE was tuned with 56 Optuna trials using validation reconstruction loss as the objective, with a guard term that penalises trials where KL divergence collapses

to zero. The selected structure uses 192 hidden units, a single layer for both the encoder and decoder, a latent dimension of 24, and a β value of 0.137 annealed linearly from zero to its target over the first 14 epochs. The full set of hyperparameters are reported in Table III.

TABLE III Tuned hyperparameters for the variational autoencoder. Common settings: batch size = 64, weight decay = 10^{-5} , patience = 25 epochs, maximum epochs = 300.

Hyperparameter	Value
Hidden size	192
Latent dimension	24
Layers (encoder & decoder)	1
Dropout	0.194
Learning rate	1.44×10^{-3}
β	0.137
KL annealing epochs	14
Optuna trials	56
Parameters	825,667

Figure 4 shows the training and validation loss curves for the VAE. The reconstruction loss decreases smoothly and plateaus before training stops, and the gap between training and validation remains small throughout, indicating no significant overfitting. The KL term rises and drops quickly during the β -annealing phase, as the encoder learns to produce informative latent codes, and stabilises once β reaches its target value. The stabilised KL value of approximately 55 nats confirms that the encoder retains substantial information about each input in its latent code, and that posterior collapse does not occur, which was why the annealing schedule was implemented.

2. Reconstruction quality

To assess what the VAE has actually learned, we examine reconstructions from the test data set. Figure 5 shows the reconstruction of a representative median difficulty test window for phosphate values in the effluent (target), influent and activated sludge aerobic reactors 5 and 7. The reconstruction signal follows the true plant state relatively closely for all variables except the influent phosphate. One reason for this can be that the influent variables are not dependent on other variables, only cycles in the data, and since the simulation data also includes artificial rain events (Gernaey *et al.*, 2014), these behaviors are difficult to learn based on the variables given to the model. The per-step squared error panels below each variables show that residual error is relatively small, with the errors being larger at specific timestep rather than throughout the entire window. This is the behavior required for the use of VAEs for anomaly detection. Further examples from the 20th, 50th and 80th

percentile of per-window reconstruction MSE is shown in Figure 10 in Appendix A.2.

3. What the VAE detects

A natural question is what the VAE is actually detecting. Because the BSM simulations is fault-free by design, with no sensor errors, no processes outside of normal operation, or no control errors, the model is not detecting anomalies in the conventional sense. Instead, the per window anomaly score measures how unusual a given test window is relative to the distribution of training windows. Windows in the edges of the distribution, which is likely windows affected by for example unusual influent loadings due to rain events, receive higher anomaly scores than windows near the center of the distribution (i.e., "normal" periods).

This setup means two things. First, the value of the anomaly score has no fault-related interpretation, it is meaningful only relative to the distribution observed during training and validation. Second, the practical value of the VAE on the clean BSM2 data is not as a fault-detector, but as a complementary model to the forecaster. It provides a signal about which windows fall outside the normal operational regime that the forecaster was trained on. Whether this signal translates to a useful fault-detection capability on a real plant with real errors, is an open question that requires labelled real-plant data to answer. The integration analysis in Section III.C quantifies the VAE's value as a complementary model to the forecaster in this simulated setting.

C. Integration

1. Score correlation

The first integration analysis tests whether the forecaster (LSTM) and the VAE, which are trained independently on different objectives, agree on which windows are difficult. Figure 6 plots the forecaster's per-window RMSE against the VAE's anomaly score for every window in the test dataset ($n = 108$ windows). The relationship is mainly positive, meaning windows that the VAE flags as atypical are also windows the forecaster performs badly on. The Pearson correlation is $r_P = +0.647$ ($p = 3.68^{-14}$) and the Spearman correlation is $r_S = +0.492$ ($p = 6.27^{-8}$), both statistically significant.

The Spearman coefficient being slightly lower than the Pearson coefficient suggests that the relationship is not strictly monotone, meaning some of the windows have high VAE anomaly score without having a corresponding high forecast error, and some of the high-error windows are not scored with a high anomaly score. This is not necessarily surprising, because the model have overlapping

VAE training on BSM2

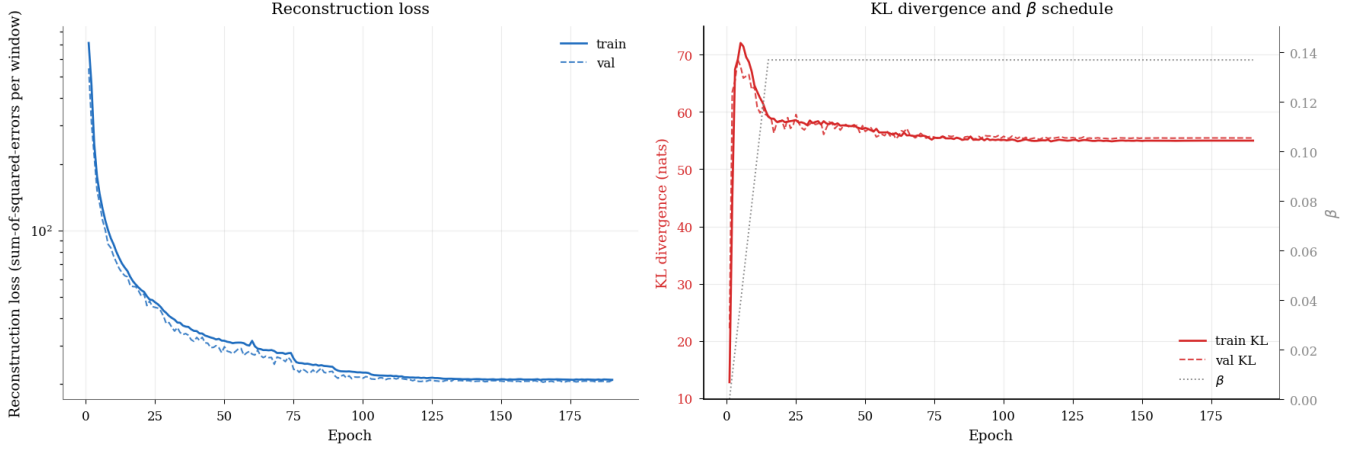


FIG. 4 VAE training on BSM2. Left: reconstruction loss (log scale), with training (solid) and validation (dashed). Right: KL divergence (red) and the β annealing schedule (grey dotted).

VAE reconstruction on BSM2 — median test window

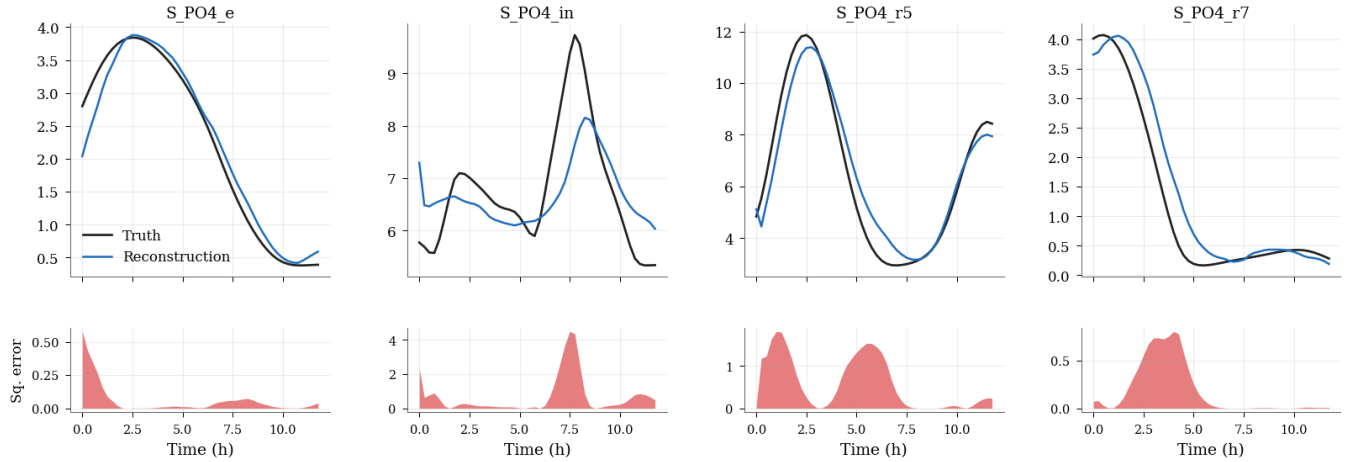


FIG. 5 VAE reconstruction on a representative median-difficulty test window. Top row: true plant signal (dark) and VAE reconstruction (blue) for the target effluent phosphate S_{PO4_e} , and phosphate in the influent, activated sludge reactor 5 and 7, respectively. Bottom row: per-step squared reconstruction error for each variable.

but not identical ideas of "hard" windows. Additionally, some of the forecast errors can come from actual unpredictability of future trajectories rather than from unusual past trajectories. The Pearson coefficient is more sensitive to high anomaly cases and captures the linear trend dominated by these outliers.

Furthermore, it is important to note that this correlation exists despite the models being trained separately on different objectives. That both models to some degree agree on which windows are difficult indicates that this difficulty is a property of the data, not because of model failure.

2. Conditional forecast accuracy

The next question is whether this relationship is operationally useful. Figure 7 sorts the test windows by VAE anomaly score, divides them into five equal-size bins, and reports the mean forecast RMSE within each bin. The RMSE rises from the cleanest bin (left) to the dirtiest (right), demonstrating that the VAE score provides a useful ranking of forecast difficulty. The cleanest 20% has a mean RMSE of 0.0819, while the dirtiest 20% has a mean RMSE of 0.2761. This shows that knowing the VAE's anomaly score for a window predicts how well the forecaster will perform on it.

Figure 8 translates the same information into a selec-

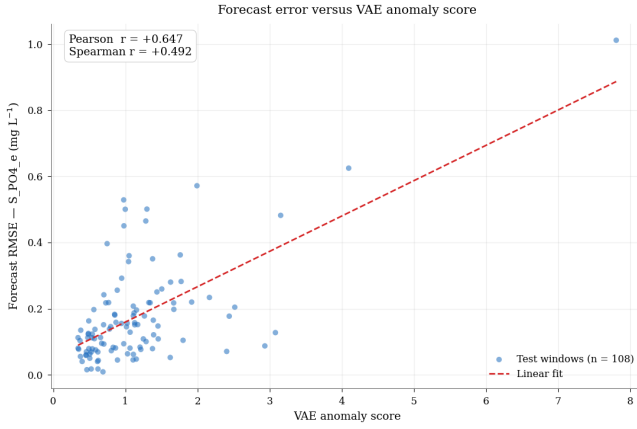


FIG. 6 Forecast error versus VAE anomaly score on the held-out test set ($n = 108$ windows). Each point is one test window; the dashed line is a linear fit. Pearson and Spearman correlations are shown on the top left.

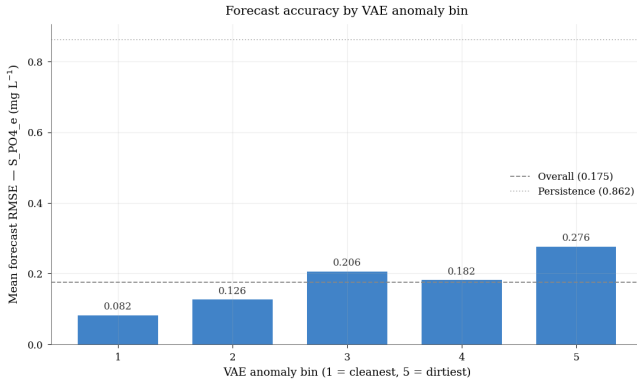


FIG. 7 Forecast accuracy by VAE anomaly bin. Test windows are sorted by VAE anomaly score and divided into five equal-size bins, from cleanest (bin 1) to dirtiest (bin 5). The persistence baseline (dotted) sits well above all bins.

tive prediction curve. The curve shows the forecast RMSE kept test windows after abstaining on the top f percent by anomaly score, as f varies from 0 to 50 percent. At $f = 20$ percent, the RMSE on the remaining 80% drops by 14.8% compared to no abstention. In a real-world plant application, this would correspond to using a forecaster as a tool 80% of the time, gaining accuracy, and reverting to alternative decision-making on the other 20% of the windows the VAE flags as atypical.

3. Operational implications

The integration analysis demonstrates that the VAE serves as a useful confidence signal for the forecaster, but the practical value of this combination depends on the context. Firstly, the integration adds value beyond normal anomaly detection only if the forecaster is part of the operational pipeline, which is a heavily studied field

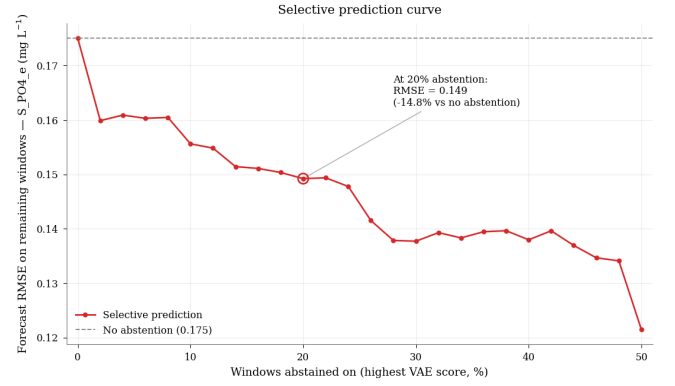


FIG. 8 Selective prediction curve. Forecast RMSE on the test windows after abstaining on the top f percent by VAE anomaly score, as f varies from 0 to 50 percent.

(Bernardelli *et al.*, 2019; Xie *et al.*, 2024). The VAE alone would tell an operator that a window looks unusual, while the integration would additionally tell the operator that the forecaster’s prediction for that window should be weighted less, or used together with other information. Secondly, as the results presented here are gained using clean simulated data without any errors, the integration value is limited to characterising distributed atypicality. On real plant data, the integration value could be higher, given real faults produce windows the forecaster cannot model, and the VAE actually captures anomalies.

D. Limitations and future work

The VAE in this project was trained and evaluated without any labelled fault data, even synthetic. It was tested to inject known synthetic faults of different types and severity into the test window, such as spikes, drift, suck values and noise, to characterise which kinds of errors the model detects best. This was not included in this work, but when combined with real data validation of the models, a fault-injected study would give a better picture of the integrations potential. The real data pathways was not chosen due to the substantial work related to the preprocessing of the data. Additionally, the real data available provided only a few usable variables with limited correlations. The data was used in early iterations of the RNNs in this project, but showed poor performance across different tests.

In addition to the points raised above, some future direction was identified:

- **Probabilistic forecasters:** Replacing the deterministic LSTM with a probabilistic alternative would enable forecasting with a predictive interval and would also allow the VAE confidence signal to be expressed intervals rather than as a binary use or not use decision.
- **Joint training:** The present integration treats the forecaster and VAE as independent components, com-

bined only at test time. A jointly trained architecture, where the forecaster's loss is weighted by the VAE's confidence, or where shared latent representations are learned, could extract more value from the input data.

- **Multi-target multi-horizon forecasting:** This study only focuses on effluent phosphate, but in principle this can be easily extended to include more targets. Additionally, comparing multiple horizons (especially longer) would give an indication on how the integration scales to different operational conditions and cyclic patterns.

IV. CONCLUSION

This project investigated two complementary uses of deep learning on plant state time series from the BSM2 wastewater treatment simulation. The first was short-horizon forecasting of effluent phosphate, and the second was unsupervised modelling of normal plant operation for anomaly detection. Three sequence architectures, LSTM, GRU, and Transformer, were compared for forecasting, a sequence VAE was trained to reconstruct normal plant-state windows. The two models were then combined in an integration analysis to test whether they agree on which windows were difficult to predict/reconstruct.

The forecasters all beat the persistence baseline by a large margin, with the LSTM performing narrowly better than the rest. The VAE learned to reconstruct normal plant windows with low error and without posterior collapse. The integration analysis revealed a positive correlation between the VAE's anomaly score and the forecaster's prediction error on the test windows despite the models being trained independently. When assessing whether abstaining on the 20% of the windows with the highest anomaly score would increase forecaster performance, the results showed a decrease in RMSE of 14.8%.

The main contribution is not showing correlation, but the demonstration that an unsupervised generative model can serve as a useful confidence signal for a separately trained predictive model. Although this is a useful comparison, the results were obtained on clean simulated data, and the integration's value on real plant data, with real faults and errors, remains an open empirical question. Transfer to real plant data, paired with probabilistic forecasters with multi-target multi-horizon objectives is the natural next step.

REFERENCES

- T. Akiba, S. Sano, T. Yanase, T. Ohta, and M. Koyama (2019), "Optuna: A next-generation hyperparameter optimization framework," in *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*, KDD '19 (Association for Computing Machinery, New York, NY, USA) p. 2623–2631.
- J. An, and S. Cho (2015), "Variational autoencoder based anomaly detection using reconstruction probability," *Special lecture on IE* **2** (1), 1–18.
- A. Bernardelli, P. Gelli, A. Manzini, S. Marsili-Libelli, S. Stancari, S. Venier, *et al.* (2019), "Real-time model predictive control of a wastewater treatment plant based on machine learning," in *Watermatex 2019* (IWA) pp. 373–380.
- S. Bowman, L. Vilnis, O. Vinyals, A. Dai, R. Jozefowicz, and S. Bengio (2016), "Generating sentences from a continuous space," in *Proceedings of the 20th SIGNLL conference on computational natural language learning*, pp. 10–21.
- K. Cho, B. Van Merriënboer, Gulçehre, D. Bahdanau, F. Bougares, H. Schwenk, and Y. Bengio (2014), "Learning phrase representations using rnn encoder–decoder for statistical machine translation," in *Proceedings of the 2014 conference on empirical methods in natural language processing (EMNLP)*, pp. 1724–1734.
- J. Chung, Ç. Gülçehre, K. Cho, and Y. Bengio (2014), "Empirical evaluation of gated recurrent neural networks on sequence modeling," *CoRR* **abs/1412.3555**, 1412.3555.
- European Council (2024), "Directive (eu) 2024/3019 of the european parliament and of the council of 27 november 2024 concerning urban wastewater treatment," .
- K. V. Gernaey, U. Jeppsson, P. A. Vanrolleghem, and J. B. Copp (2014), *Benchmarking of control strategies for wastewater treatment plants* (IWA publishing).
- F. A. Gers, J. Schmidhuber, and F. Cummins (2000), "Learning to forget: Continual prediction with lstm," *Neural Computation* **12** (10), 2451–2471.
- M. Henze, W. Gujer, T. Mino, T. Matsuo, M. C. Wentzel, G. v. R. Marais, and M. C. M. Van Loosdrecht (1999), "Activated sludge model no.2d, asm2d," *Water Science and Technology* **39** (1), 165–182, <https://iwaponline.com/wst/article-pdf/39/1/165/26975/165.pdf>.
- I. Higgins, L. Matthey, A. Pal, C. Burgess, X. Glorot, M. Botvinick, S. Mohamed, and A. Lerchner (2017), "beta-VAE: Learning basic visual concepts with a constrained variational framework," in *International Conference on Learning Representations*.
- S. Hochreiter, and J. Schmidhuber (1997), "Long short-term memory," *Neural Computation* **9** (8), 1735.
- U. Jeppsson, M.-N. Pons, I. Nopens, J. Alex, J. Copp, K. Gernaey, C. Rosen, J.-P. Steyer, and P. Vanrolleghem (2007), "Benchmark simulation model no 2: general protocol and exploratory case studies," *Water Science and Technology* **56** (8), 67–78, <https://iwaponline.com/wst/article-pdf/56/8/67/438147/67.pdf>.
- D. P. Kingma, and J. Ba (2014), "Adam: A method for stochastic optimization," arXiv preprint arXiv:1412.6980.
- D. P. Kingma, and M. Welling (2013), "Auto-encoding variational bayes," arXiv preprint arXiv:1312.6114.
- D. P. Kingma, and M. Welling (2019), "An introduction to variational autoencoders," *Foundations and Trends in Machine Learning* **12** (4), 307–392.
- A. Paszke, S. Gross, F. Massa, A. Lerer, J. Bradbury, G. Chanan, T. Killeen, Z. Lin, N. Gimelshein, L. Antiga, A. Desmaison, A. Kopf, E. Yang, Z. DeVito, M. Raison, A. Tejani, S. Chilamkurthy, B. Steiner, L. Fang, J. Bai, and S. Chintala (2019), "Pytorch: An imperative style, high-performance deep learning library," in *Advances in Neural Information Processing Systems*, Vol. 32, edited by H. Wallach, H. Larochelle, A. Beygelzimer, F. d'Alché-Buc,

- E. Fox, and R. Garnett (Curran Associates, Inc.).
- S. Raschka, Y. H. Liu, and V. Mirjalili (2022), *Machine Learning with PyTorch and Scikit-Learn: Develop machine learning and deep learning models with Python* (Packt Publishing Ltd).
- M. Schuster, and K. Paliwal (1997), “Bidirectional recurrent neural networks,” *IEEE Transactions on Signal Processing* **45** (11), 2673–2681.
- V. Smil (2000), “Phosphorus in the environment: Natural flows and human interferences,” *Annual Review of Environment and Resources* **25** (Volume 25, 2000), 53–88.
- K. Solon, X. Flores-Alsina, C. Kazadi Mbamba, D. Ikumi, E. Volcke, C. Vaneeckhaute, G. Ekama, P. Vanrolleghem, D. Batstone, K. Gernaey, and U. Jeppsson (2017), “Plant-wide modelling of phosphorus transformations in wastewater treatment systems: Impacts of control and operational strategies,” *Water Research* **113**, 97–110.
- S. B. Taieb, and A. F. Atiya (2016), “A bias and variance analysis for multistep-ahead time series forecasting,” *IEEE Transactions on Neural Networks and Learning Systems* **27** (1), 62–76.
- A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. u. Kaiser, and I. Polosukhin (2017), “Attention is all you need,” in *Advances in Neural Information Processing Systems*, Vol. 30, edited by I. Guyon, U. V. Luxburg, S. Bengio, H. Wallach, R. Fergus, S. Vishwanathan, and R. Garnett (Curran Associates, Inc.).
- Y. Xie, Y. Chen, Q. Wei, and H. Yin (2024), “A hybrid deep learning approach to improve real-time effluent quality prediction in wastewater treatment plant,” *Water Research* **250**, 121092.

Appendix A: Supplementary results

1. Architecture comparison plots

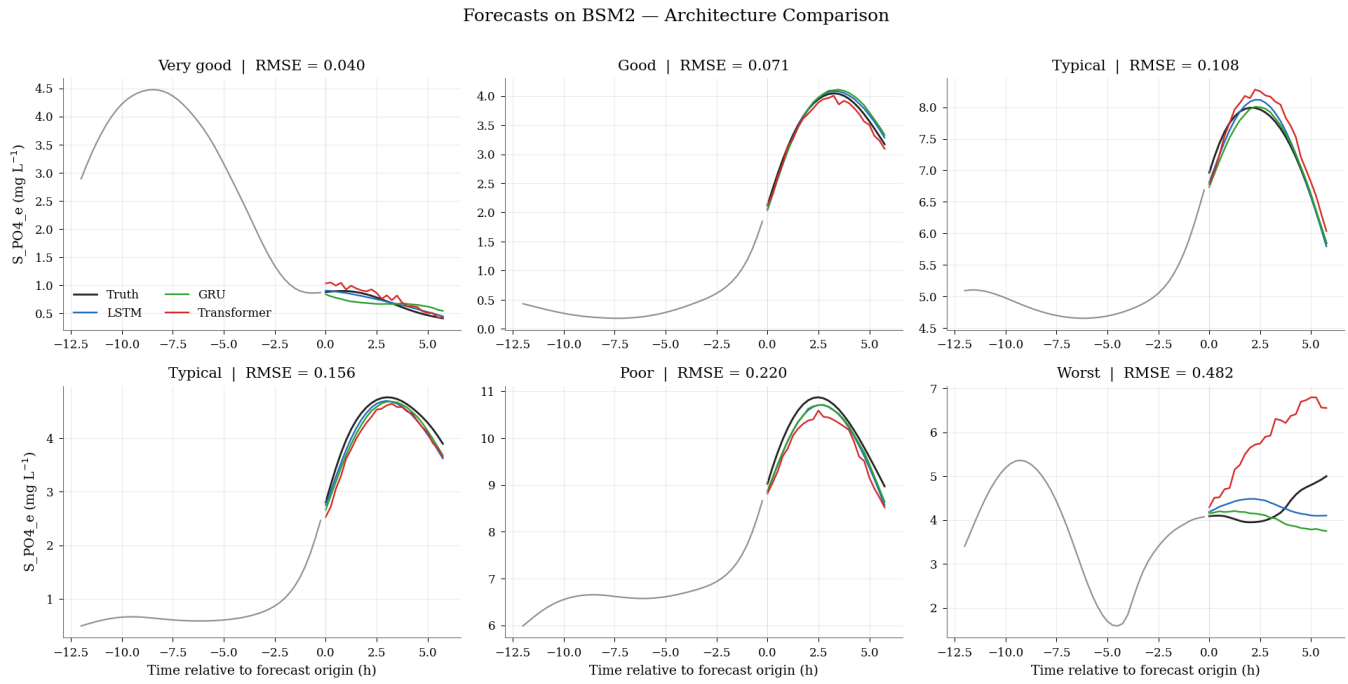


FIG. 9 Forecast trajectories for the LSTM (blue), GRU (green), and Transformer (red) on the same six BSM2 test windows shown in Figure 2.

2. Additional reconstruction examples

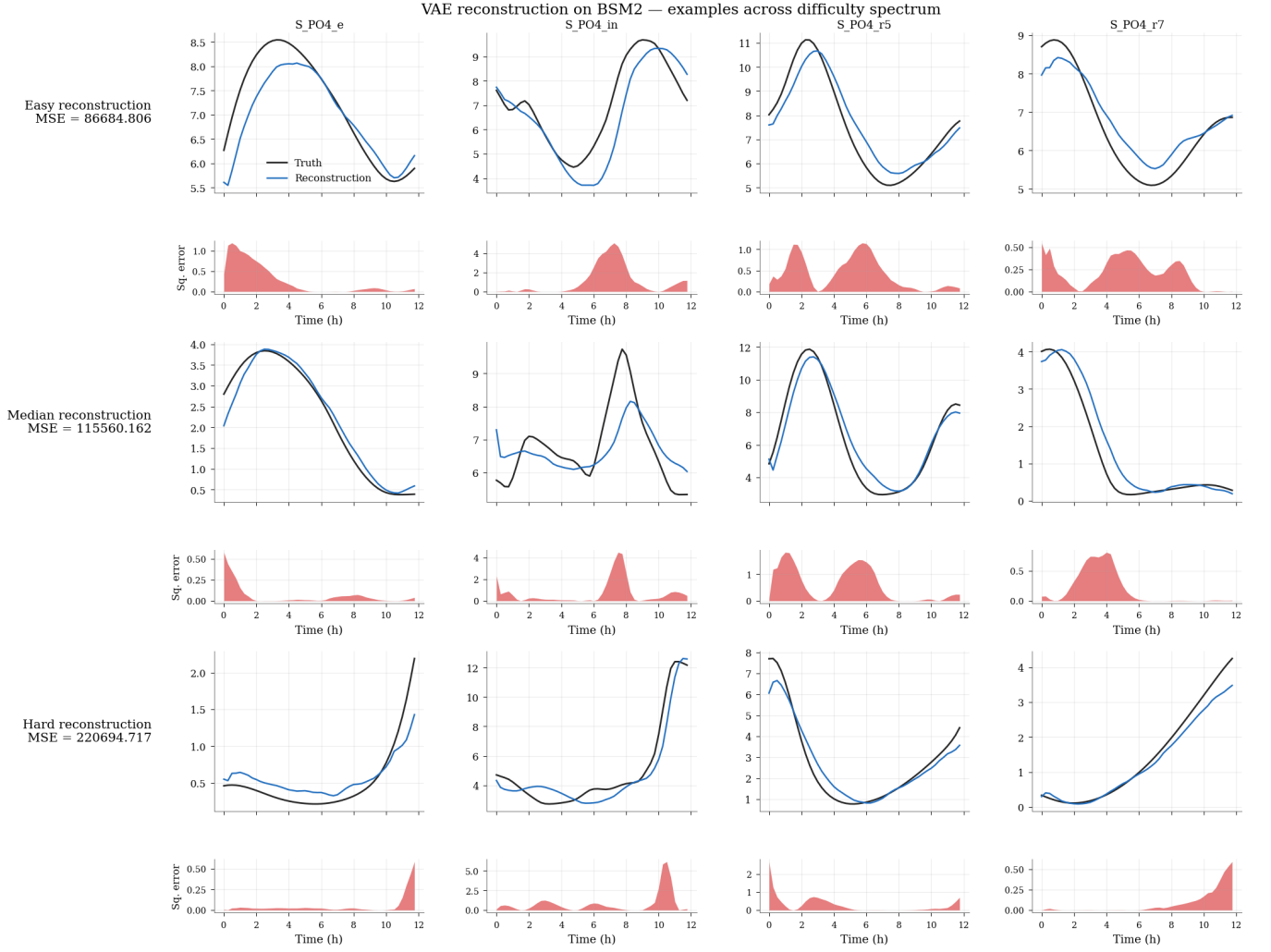


FIG. 10 VAE reconstruction across the difficulty spectrum. Three test windows are shown at the 20th, 50th, and 80th percentile of per-window reconstruction MSE (top to bottom rows, labelled on the left). For each window, the true signal (dark) and the VAE reconstruction (blue) are shown for the four variables introduced in Figure 5, with per-step squared error plotted below each.

Appendix B: Use of AI/LLM Tools

A.1 Tools used

- Claude (Claude Opus 4.7, accessed via claude.ai, April-June 2026)
- GitHub Copilot (VS Code extension, used throughout the project)

A.2 Text contributions

Section	Level	Notes
Abstract	3	Draft created by LLM based on conclusion and introduction, rewrote with values by me.
Introduction	2	LLM provided section structure and rephrased paragraphs.
Methods (theoretical foundations)	1	LLM provided equation formats for latex, section structure and minor phrasing.
Methods (BSM2 description)	1	LLM provided equation formats for latex, section structure and minor phrasing.
Methods (implementation)	1	LLM provided equation formats for latex, section structure and minor phrasing.
Results (forecasting)	1	LLM provided section structure.
Results (VAE)	1	LLM provided section structure.
Results (integration)	1	LLM provided section structure.
Limitations / future work	2	LLM provided section structure and paragraph phrasing.
Conclusion	1	LLM provided section structure.

A.3 Code contributions

File / notebook	Level	Description
MODEL_1_FORECASTER.ipynb	4	See note below
MODEL_2_VAE.ipynb	4	See note below
MODEL_3_INTEGRATED.ipynb	4	See note below
Optuna_viewer.ipynb	4	Generated by Claude, tested by me.

a. *Note on the three model notebooks.* The three project notebooks (forecasting, VAE, integration) were developed by me iteratively with substantial Claude assistance throughout. Model architectures, training loops, hyperparameter search, evaluation code, and the integration analyses were drafted and debugged by Claude, all based on my specifications and input, and over multiple rounds of testing and revision. All code was executed and verified by me. Due to a change in project focus on a late stage, having Claude draft code is substantially more effective.